

Failure (a dead connection, and a real container)

— Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `db_failure_and_docker_practice.py`: error handling you never saw fail is error handling you only hope works. This exercise breaks the database on purpose, then ships the app in a container and proves no secret went in with it. **If not:** the Build Task's "fake internal error → 500" and "missing env var → clear error at startup" test cases, and its Dockerfile, would all be first attempts.

Read both depths — they're not "wrong, right," they're 2 real, valid ways to handle the same two problems (a broken database connection, and a real container), with real tradeoffs between them. For Docker, put a `requirements.txt` in `practice/` (`fastapi` , `uvicorn[standard]`) and run `docker build` from there.

Basic Version

Approach 1 — the direct way

```
# practice/db_failure_and_docker_practice.py
import logging
import sqlite3
from fastapi import FastAPI, HTTPException

logging.basicConfig(level=logging.INFO)

app = FastAPI()
DB_PATH = "/no/such/path/runs.db"    # broken on purpose

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/runs")
def get_runs():
    try:
        conn = sqlite3.connect(DB_PATH)
        rows = conn.execute("SELECT * FROM runs").fetchall()
```

```

    return rows
except Exception as e:
    logging.error("GET /runs failed: %s", e)
    raise HTTPException(status_code=500, detail="Something went wrong.")

```

Expected output:

```

$ curl -i localhost:8000/runs
HTTP/1.1 500 Internal Server Error
{"detail":"Something went wrong."}

```

Server terminal:

```

ERROR:root:GET /runs failed: unable to open database file

```

No traceback reaches the client, and the server itself doesn't crash — `/health` still responds normally. The real reason only appears in *your* terminal. The connect sits inside the `try` on purpose: `sqlite3.connect()` to a missing folder fails right away, not on the first query.

Revision from Doc01 (Basic): this is the simplest logging setup from Doc01 — one `logging.basicConfig(...)` line and one `logging.error(...)` call. `ERROR` is the right level: one operation failed, but the rest of the app keeps working. Without that line, the client sees "Something went wrong." and so do you — nobody can find out why.

```

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "db_failure_and_docker_practice:app", \
    "--host", "0.0.0.0", "--port", "8000"]

```

Expected output:

```

$ docker build -t my-api .
$ docker run -p 8000:8000 -e OPENAI_API_KEY=sk-test-123 my-api
$ curl localhost:8000/health
{"status":"ok"}

```

This satisfies the exercise's core requirements. It opens a new connection on every request, has no `.dockerignore`, and doesn't check required settings at startup — all addressed below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — one guarded connection at startup, plus a `.dockerignore`

Story: real apps open their database connection once, at startup, like `sqlite_persistence` did — but then a bad path would crash the whole app before `/health` could even answer. Guarding the connect keeps the app up and fails only the routes that need the database. A `.dockerignore` keeps `.env` out of the image, and `docker history` proves it. **If not:** one wrong path would take down every route, and "no secret in the image" would be a hope, not a checked fact.

```
# practice/db_failure_and_docker_practice.py
import sqlite3
from fastapi import FastAPI, HTTPException

app = FastAPI()

# why: connect() itself can fail – guard it, or the app never starts
try:
    conn = sqlite3.connect("/no/such/path/runs.db", check_same_thread=False)
    conn.execute("CREATE TABLE IF NOT EXISTS runs (id INTEGER PRIMARY KEY)")
except sqlite3.OperationalError:
    # how: None means "no database" – routes check for it below
    conn = None

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/runs")
def get_runs():
    if conn is None:
        raise HTTPException(status_code=500, detail="Something went wrong.")
    try:
        return conn.execute("SELECT * FROM runs").fetchall()
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")
```

Expected output:

```
$ curl localhost:8000/health
{"status":"ok"}
$ curl -i localhost:8000/runs
HTTP/1.1 500 Internal Server Error
{"detail":"Something went wrong."}
```

Even though the database path is broken from the start, the app still starts and `/health` still works — only the routes that actually need the database fail, and fail cleanly.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "db_failure_and_docker_practice:app", \
    "--host", "0.0.0.0", "--port", "8000"]
```

```
# .dockerignore
.env
.git
__pycache__/
*.pyc
.venv/
*.db
```

Expected output:

```
$ docker build -t my-api .
$ docker run -p 8000:8000 -e OPENAI_API_KEY=sk-test-123 my-api
$ docker history --no-trunc my-api | grep "sk-test-123"
(no output – the key was only passed with -e at run time,
so it appears in no layer of the image)
```

`.dockerignore` stops `COPY . .` from ever picking up `.env` or `.git`, and the `grep` proves the real key value is nowhere in the image. Run this check on every image before it ships.

Approach 2 — a startup check that stops the container if a required env var is missing

Story: a container that starts without its API key looks "up" but fails on every real request — the worst kind of failure, because nothing tells you why. Checking at startup turns it into one clear error the moment the container starts. **If not:** the Build Task's "docker run with a required env var missing" test case would show a silent, half-working container instead of a clear failure.

```
# practice/db_failure_and_docker_practice.py
import os
import sqlite3
from fastapi import FastAPI, HTTPException

class MissingConfigError(Exception):
    """Same custom error as Doc01's exceptions.py."""

# why: fail at startup, not on the first real request
# how: os.getenv returns None when the variable isn't set
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
if not OPENAI_API_KEY:
    raise MissingConfigError(
        "OPENAI_API_KEY is not set – pass it with docker run -e"
    )

app = FastAPI()

try:
    conn = sqlite3.connect("runs.db", check_same_thread=False)
    conn.execute("CREATE TABLE IF NOT EXISTS runs (id INTEGER PRIMARY KEY)")
except sqlite3.OperationalError:
    conn = None

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/runs")
def get_runs():
    if conn is None:
        raise HTTPException(status_code=500, detail="Something went wrong.")
    try:
        return conn.execute("SELECT * FROM runs").fetchall()
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")
```

Expected output with the required env var missing:

```
$ docker run -p 8000:8000 my-api
Traceback (most recent call last):
...
MissingConfigError: OPENAI_API_KEY is not set – pass it with docker run -e
```

```
$ echo $?
1
```

The container exits at once with a clear, specific error — not a silent hang, and not a container that looks "up" but can't do anything.

Revision from Doc01: this is the exact "required value: check at startup, fail loudly" rule from Doc01's `.env` topic — nothing changes just because the value now arrives through `docker run -e` instead of a `.env` file. `os.environ["OPENAI_API_KEY"]` would also crash at startup, but with a bare `KeyError: 'OPENAI_API_KEY'`; the named `MissingConfigError` also tells whoever reads the container logs how to fix it. In a real project, don't redefine the class here — import it from your Doc01 files.

Difference from Basic: Approach 1 opens one connection at startup and guards it, so a broken path fails only the routes that need it instead of the whole app, adds `.dockerignore` so `COPY . .` can never pick up `.env`, and proves it with `docker history --no-trunc | grep .` Approach 2 adds a second, different failure worth testing: a required setting that's missing at container start — and shows why an explicit check with a named error beats a plain `os.getenv(key)` that silently returns `None`.

Which one should you actually write? Both, together — they're the minimum any real Dockerized API needs: guard the connection, add `.dockerignore`, check the image with `docker history`, and fail loudly on a missing required env var. The Build Task's Dockerfile, `.dockerignore` and startup check are exactly these two approaches, pointed at `api.main:app` instead of this practice file.